Solution Requirements

Phase 2: Requirement Analysis

1. Purpose and Scope

The requirements define the behaviours the current application is intended to provide. They are separated into functional requirements, which describe user-visible capabilities, and non-functional requirements, which describe qualities such as clarity, reliability and safe configuration. Requirements are mapped to source areas where known, but a source file alone does not prove that a feature has passed end-to-end testing.

2. Functional Requirements

FR-01: Collect trip details

The frontend should accept a starting city, destination, travel dates, budget, traveller count, interests and travel style. Inputs should be passed to the backend in a structured request.

FR-02: Generate an itinerary

The backend should call the configured Groq model and return a structured itinerary. The output should include a trip summary, daily activities, estimated costs and travel tips in the expected schema.

FR-03: Validate model output

The AI service should parse the response and check required fields before the frontend treats it as a usable itinerary. Invalid output should produce a clear error instead of corrupting the page.

FR-04: Persist and retrieve trips

The application should save trip and itinerary data to Supabase, list saved trips, retrieve a selected trip and delete a trip together with its associated itinerary.

FR-05: Support external place discovery

The hotel endpoint should return a destination-based Google Maps search URL. The restaurant endpoint should include a selected food-preference term when provided. The system must not represent these links as verified inventory.

FR-06: Print or save a plan

The frontend should invoke the browser print interface so that the user can print or select Save as PDF.

3. Non-Functional Requirements

- Usability: fields and actions should be understandable to a first-time user.
- Maintainability: API routes, AI logic, schemas and database setup should remain separated.
- Reliability: model and database failures should not silently appear as success.
- Security: service credentials should be private and server-side.
- Portability: the frontend should run in a modern browser.
- Transparency: estimates and external searches should be described accurately.

4. Acceptance Criteria

A requirement should be marked as passed only after the deployed flow has been tested. For example, successful generation requires a valid request, visible itinerary, and appropriate handling of failure cases. Persistence requires saving, reloading, retrieving and deleting a record. Place search requires opening the expected Google Maps query, not proof that each listing is correct.

5. Constraints and Assumptions

The application depends on internet access, an available AI provider, valid environment configuration and Supabase availability. Model quotas or network outages may interrupt generation. Costs and travel suggestions are estimates, and the user remains responsible for checking details before travel.

6. Traceability

The frontend is primarily implemented in frontend/src/App.tsx. Backend routing and orchestration are in backend/main.py, AI generation in backend/ai_service.py, request/response models in backend/schemas.py and database setup in backend/supabase_client.py. Use these locations when reviewing the source archive.